

Guía de Instalación en Nuevo Nodo

Guía de Instalación / Replicación en otro Nodo — FastChat DJ (IA + RAG)

Guía práctica y reproducible para instalar el sistema **completo** (aplicación Django + Weaviate + IA/RAG + cotizador) en un servidor nuevo. Objetivo: que el cliente pueda **replicar todo** en otro nodo/servidor. Todos los secretos aparecen como **placeholders** (`<...>`). Sustituirlos por los valores reales; nunca commitarlos.

Índice

- [Requisitos](#)
- [A. Instalar la app Django \(Web IA\)](#)
- [B. Instalar Docker + Weaviate](#)
- [C. Configurar embeddings e IA](#)
- [D. Provisionar y usar](#)
- [E. Checklist de verificación](#)

Requisitos

Sistema operativo y paquetes

Componente	Versión / nota
SO	Debian 12 (bookworm) , x86_64
Python	3.11 (el venv del servidor de referencia usa Python 3.11)
PostgreSQL	17 (funciona 15+; el servidor real usa 17)
Redis	5.x (opcional pero recomendado para escalar Channels)
Docker	29.x (Docker Engine + plugin <code>compose</code>) — para Weaviate
nginx	reverse proxy en :80

Componente	Versión / nota
Weaviate	imagen 1.28.4 (o compatible v1.28.x)
ffmpeg	opcional — solo si se usa la funcionalidad de voz (Whisper)

Recursos mínimos

- El servidor de referencia corre con **1 vCPU / 3.8 GiB RAM**. Con esos recursos es **imprescindible** una partición **swap** (se usó **4 GB**) como red de seguridad, y bajar la agresividad del swapping:

```
# Crear swap de 4 GB (si no existe)
sudo fallocate -l 4G /swapfile
sudo chmod 600 /swapfile
sudo mkswap /swapfile
sudo swapon /swapfile
echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab

# Reducir swappiness a 10 (usar swap solo cuando sea necesario)
echo 'vm.swappiness=10' | sudo tee /etc/sysctl.d/99-swappiness.conf
sudo sysctl -w vm.swappiness=10
```

- Weaviate se limita en memoria con `GOMEMLIMIT` (1 GB en el servidor de referencia) — ver el `docker-compose.yml` de la sección B.
- Los **embeddings del RAG son externos (Gemini)**: no cargan CPU/RAM del servidor. No se usan modelos locales de embeddings en runtime.

Paquetes base del sistema

```
sudo apt update
sudo apt install -y python3.11 python3.11-venv python3-pip \
    postgresql postgresql-contrib redis-server nginx git curl ca-certificates \
    build-essential libpq-dev
# Opcional (voz):
sudo apt install -y ffmpeg
```

A. Instalar la app Django (Web IA)

A.1. Clonar el repositorio y crear el virtualenv

```
sudo mkdir -p /home/fastchatdj
sudo chown "$USER" /home/fastchatdj
git clone <URL_DEL_REPO_SIN_TOKEN> /home/fastchatdj
cd /home/fastchatdj
```

```
# El servidor de referencia usa un venv en /home/venv
python3.11 -m venv /home/venv
source /home/venv/bin/activate
python -m pip install --upgrade pip
```

Importante — clonar sin token en la URL: nunca uses una URL de tipo `https://<token>@github.com/...`. Usa una **deploy key SSH** o un *credential helper*. (Ver `SEGURIDAD.md`.)

A.2. Instalar dependencias con las versiones fijadas

El `requirements.txt` cubre el núcleo Django + IA. **Las dependencias del RAG (Weaviate + langchain-openai + langchain-anthropic) NO están en `requirements.txt` y deben instalarse aparte con sus versiones fijadas.**

```
# requirements.txt está codificado en UTF-16; pip lo lee bien, pero si lo procesas
# manualmente conviértelo: iconv -f UTF-16 -t UTF-8 requirements.txt > /tmp/req.txt
pip install -r requirements.txt

# Dependencias del RAG / Ollama / Claude (NO en requirements.txt) – versiones
# fijadas:
pip install \
    weaviate-client==4.21.0 \
    langchain-weaviate==0.0.6 \
    langchain-openai==0.2.14 \
    langchain-core==0.3.67 \
    openai==1.93.0 \
    langchain-anthropic
```

⚠ Versiones críticas (no subir)

Paquete	Versión	Motivo
langchain-core	0.3.67	Subir a 1.x rompe todo el ecosistema 0.3.x
langchain-openai	0.2.14	La 1.x arrastra langchain-core 1.x (rompe tool-calling de Ollama)
openai	1.93.0	Compatibilidad con langchain 0.3.x
numpy	serie 1.26.x (no 2.x)	numpy 2.x rompe numba / whisper. requirements.txt fija 1.26.2 ; en el servidor se usó 1.26.4 . Nunca subir a 2.x.
weaviate-client	4.21.0	

Paquete	Versión	Motivo
		API v4 usada por <code>weaviate_rag.py</code>
<code>langchain-weaviate</code>	0.0.6	—

Tras instalar, verificar que `numpy` sigue en 1.26.x (algunas dependencias lo intentan subir):

```
python -c "import numpy; print(numpy.__version__)" # debe ser 1.26.x
# si subió: pip install 'numpy==1.26.4'
```

A.3. Configurar `credenciales.json`

El proyecto lee la configuración sensible de `credenciales.json` en la raíz del repo (fuera de git). Usar `credenciales_template.json` como base:

`credenciales_template.json` (referencia de claves requeridas):

```
{
  "POSTGRES_HOST": "",
  "POSTGRES_PORT": "",
  "POSTGRES_PASSWORD": "",
  "POSTGRES_DBNAME": "",
  "USE_SSL": false,
  "SECRET_KEY": "",
  "EMAIL_HOST_USER": "",
  "EMAIL_HOST_PASSWORD": "",
  "WKHTMLOPDF_CMD": "",
  "GMAP_API_KEY": "",
  "CHATBOT_ERROR_NOTIFY_EMAILS": ["<correo_notificaciones>"],
  "DEBUG": false
}
```

Crear el archivo real con permisos restrictivos:

```
cp credenciales_template.json credenciales.json
# Editar credenciales.json con los valores reales
chmod 600 credenciales.json
chown <usuario_del_servicio>:<usuario_del_servicio> credenciales.json
```

Seguridad: `credenciales.json` debe ser `600` (solo el dueño lee/escribe). Ver `SEGURIDAD.md`.

A.4. PostgreSQL: crear base de datos y usuario

```
sudo -u postgres psql <<'SQL'
CREATE USER fastchat WITH PASSWORD '<POSTGRES_PASSWORD>';
```

```
CREATE DATABASE dbfastchat OWNER fastchat;
GRANT ALL PRIVILEGES ON DATABASE dbfastchat TO fastchat;
SQL
```

Poblar en `credenciales.json`: `POSTGRES_HOST` (p.ej. `127.0.0.1`), `POSTGRES_PORT` (`5432`), `POSTGRES_DBNAME` (`dbfastchat`), `POSTGRES_PASSWORD`.

El settings del proyecto usa `ATOMIC_REQUESTS = True`, `LANGUAGE_CODE = 'es-ec'`, `TIME_ZONE = 'America/Guayaquil'`, `ALLOWED_HOSTS = ["*"]` (restringir en producción — ver `SEGURIDAD.md`).

A.5. Verificar apps instaladas, migrar y estáticos

Asegurar que `INSTALLED_APPS` (en `fastchatdj/settings.py`) incluye las apps de esta fase:

```
INSTALLED_APPS = [
    # ...
    'agents_ai.apps.AgentsAiConfig',
    'cotizador',  # <-- añadir si no está (la app del cotizador médico)
    # ...
]
```

```
source /home/venv/bin/activate
cd /home/fastchatdj
python manage.py migrate
python manage.py collectstatic --noinput
```

La app `cotizador` trae su migración `0001_initial`; `crm` incluye la migración `0003` que agrega el proveedor Ollama a `PROVEEDOR_CHOICES`. La capa RAG (`agents_ai/indexador_conocimiento.py`, cambios en `agente_consultor.py`, `providers/`) **no requiere migraciones** (defaults por vista/formulario).

A.6. Servicio systemd (Daphne) detrás de nginx

`/etc/systemd/system/fastchatdj.service`:

```
[Unit]
Description=FastChat DJ (Daphne ASGI)
After=network.target postgresql.service

[Service]
User=<usuario_del_servicio>
Group=<usuario_del_servicio>
WorkingDirectory=/home/fastchatdj
EnvironmentFile=/etc/fastchatdj.env
ExecStart=/home/venv/bin/daphne -u /run/fastchatdj.sock \
```

```
--application-close-timeout 180 fastchatdj.asgi:application
Restart=on-failure
RuntimeDirectory=fastchatdj

[Install]
WantedBy=multi-user.target
```

`RuntimeDirectory=fastchatdj` crea `/run/fastchatdj` con permisos correctos; el socket vive en `/run/fastchatdj.sock`. El `EnvironmentFile=/etc/fastchatdj.env` inyecta las variables de entorno (Weaviate, cédula, etc. — ver sección C). En producción, `User` debe ser un **usuario sin privilegios** (no root).

`/etc/nginx/sites-available/fastchatdj`:

```
server {
    listen 80;
    server_name <IP_O_DOMINIO>;

    location /static/ { alias /home/fastchatdj/static/; }
    location /media/ { alias /home/fastchatdj/media/; }

    location / {
        proxy_pass http://unix:/run/fastchatdj.sock;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;           # WebSocket
        proxy_set_header Connection "upgrade";
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

Activar y arrancar:

```
sudo ln -sf /etc/nginx/sites-available/fastchatdj /etc/nginx/sites-enabled/
fastchatdj
sudo nginx -t && sudo systemctl reload nginx

sudo systemctl daemon-reload
sudo systemctl enable --now fastchatdj.service
sudo systemctl status fastchatdj.service
```

Recordatorio operativo: el chat web usa un proceso persistente. **Cada cambio de código requiere reiniciar Daphne:** `sudo systemctl restart fastchatdj.service`.

B. Instalar Docker + Weaviate

B.1. Instalar Docker Engine + Compose (Debian 12)

```
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/debian/gpg | \
  sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/
  docker.gpg] \
  https://download.docker.com/linux/debian bookworm stable" | \
  sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-compose-plugin
sudo systemctl enable --now docker
docker --version
```

B.2. docker-compose.yml de Weaviate

Crear el directorio y los archivos:

```
sudo mkdir -p /home/weaviate
cd /home/weaviate
```

/home/weaviate/.env:

```
WEAVIATE_API_KEY=<TU_WEAVIATE_API_KEY>
```

/home/weaviate/docker-compose.yml:

```
services:
  weaviate:
    image: cr.weaviate.io/semitechnologies/weaviate:1.28.4
    container_name: weaviate
    restart: unless-stopped
    command:
      - --host
      - 0.0.0.0
      - --port
      - "8080"
      - --scheme
      - http
    ports:
      # Publicar SOLO en localhost (nunca 0.0.0.0) – la app se conecta por 127.0.0.1
      - "127.0.0.1:8080:8080"
      - "127.0.0.1:50051:50051"
    environment:
      # Persistencia
      PERSISTENCE_DATA_PATH: "/var/lib/weaviate"
      # Sin módulos: los vectores los provee la app (BYO vectors / vectorizer=none)
      DEFAULT_VECTORIZER_MODULE: "none"
      ENABLE_MODULES: ""
```

```

# Autenticación por API key
AUTHENTICATION_APIKEY_ENABLED: "true"
AUTHENTICATION_APIKEY_ALLOWED_KEYS: "${WEAVIATE_API_KEY}"
AUTHENTICATION_APIKEY_USERS: "fastchatdj@local"
AUTHORIZATION_ADMINLIST_ENABLED: "true"
AUTHORIZATION_ADMINLIST_USERS: "fastchatdj@local"
# Deshabilitar acceso anónimo
AUTHENTICATION_ANONYMOUS_ACCESS_ENABLED: "false"
# Multi-tenancy: creación automática de tenants
AUTOSHEMA_ENABLED: "true"
# Límite de memoria de Go (servidor pequeño)
GOMEMLIMIT: "1GiB"
QUERY_DEFAULTS_LIMIT: "25"
volumes:
  - weaviate_data:/var/lib/weaviate

volumes:
  weaviate_data:

```

Levantar y verificar:

```

cd /home/weaviate
docker compose up -d
docker compose ps

# Verificar readiness (requiere la API key):
curl -s -H "Authorization: Bearer <TU_WEAVIATE_API_KEY>" \
  http://127.0.0.1:8080/v1/.well-known/ready -o /dev/null -w "%{http_code}\n"
# Debe imprimir: 200

```

Aislamiento de red: los puertos se publican como 127.0.0.1:8080 y 127.0.0.1:50051 — Weaviate **nunca** queda expuesto a internet. La app Django se conecta por `localhost` con la API key. Ver `SEGURIDAD.md`.

B.3. Cómo lo usa la aplicación

`agents_ai/weaviate_rag.py` lee la configuración de conexión, **en este orden**:

1. Variables de entorno: `WEAVIATE_HOST` (default `127.0.0.1`), `WEAVIATE_HTTP_PORT` (`8080`), `WEAVIATE_GRPC_PORT` (`50051`), `WEAVIATE_API_KEY`.
2. Si no están en el entorno, las busca en `/home/weaviate/.env`.

La colección `Conocimiento` (multi-tenant, `vectorizer=none`) se **crea automáticamente** la primera vez que se indexa (`ensure_schema`). No hay que crearla a mano.

C. Configurar embeddings e IA

C.1. Variables de entorno del servicio (`/etc/fastchatdj.env`)

Este archivo lo consume el servicio `systemd` (`EnvironmentFile`). Evita problemas de escape de caracteres (`$`) en las credenciales.

`/etc/fastchatdj.env` :

```
# --- Weaviate (RAG) ---
WEAVIATE_HOST=127.0.0.1
WEAVIATE_HTTP_PORT=8080
WEAVIATE_GRPC_PORT=50051
WEAVIATE_API_KEY=<TU_WEAVIATE_API_KEY>

# --- Cotizador: API de cédula (Vida Nueva / Broktech) ---
VIDANUEVA_USER=<USUARIO_BROKTECH>
VIDANUEVA_PASS=<PASSWORD_BROKTECH>
```

```
sudo chmod 600 /etc/fastchatdj.env
sudo chown root:root /etc/fastchatdj.env
sudo systemctl restart fastchatdj.service
```

Notas: - `WEAVIATE_API_KEY` debe coincidir con la del `docker-compose.yml` de Weaviate. - `VIDANUEVA_USER/PASS` son las credenciales de Broktech para la captura por cédula. Si no se ponen, el sistema cae directo a la API de Cotimédica (fallback).

C.2. API Key de Gemini (embeddings del RAG) — por perfil

Los **embeddings del RAG** los genera **Google Gemini** (`models/gemini-embedding-001`). La key **no** va en variables de entorno: se registra **por perfil** desde el panel, como una **API Key de proveedor Gemini** (`proveedor=2`).

- El motor (`agente_consultor._resolver_embed_key` y `indexador_conocimiento._resolver_gemini_key`) toma la key Gemini **activa** del perfil (`estado=True`) de mayor id.
- **Recomendación operativa:** mantener **una sola** key Gemini válida y activa por perfil. Si hay varias y alguna es inválida, el sistema puede resolver la inválida y el RAG queda sin embeddings (fue un incidente real de esta fase). Deshabilitar (no borrar) las keys que no sirvan.

C.3. API Keys de los LLM (chat) — desde el panel

El **modelo de chat** (distinto de los embeddings) se configura **desde la interfaz**:

1. En el perfil/agente, registrar una **API Key** del proveedor deseado:
 - **Gemini** (`proveedor=2`), **OpenAI** (`3`), **Claude** (`4`), **Ollama Cloud** (`5`).

2. En la **Configuración del agente**, el dropdown **Modelo** trae la **lista viva** de modelos del proveedor (acción `listar_modelos`, con caché de 30 min). Elegir el modelo y guardar el agente.

Ollama Cloud: la API key se usa contra `https://ollama.com/v1`. El modelo de referencia del caso Vida Buena es `gemma4:31b`. **Embeddings vs. chat:** Ollama y Claude **no** proveen embeddings; por eso el RAG siempre usa Gemini para embeddings, aunque el chat use otro proveedor.

D. Provisionar y usar

Flujo completo desde el panel (sin SSH):

1. **Crear el perfil de la empresa** (`PerfilNegocioIA`) y su **API Key Gemini** (para embeddings).
2. **Crear el agente** desde el panel. Al guardarlo, el sistema **auto-provisiona su tenant Weaviate** (`agente_<id>`) — nace con la infraestructura RAG lista.
3. **Configurar el modelo de chat:** registrar la API Key del proveedor (Ollama/Gemini/...) y elegir el modelo del dropdown de lista viva.
4. **Subir documentos** en la pestaña **Conocimiento** → botón “**Subir documento**”:
 - Formatos: **PDF, DOCX, XLSX/XLS, CSV, JSON, TXT**. Máx **20 MB**.
 - Al subir, el documento se guarda y se **indexa automáticamente** al tenant del agente (extracción de texto → troceo ~1000 chars → embeddings Gemini → Weaviate).
5. **Reindexar** cuando haga falta: botón “**Reindexar al RAG**” (acción `rag_reindex`) reconstruye las fuentes del panel en el tenant. El panel muestra el n° de fragmentos y las fuentes indexadas.
6. **Editar el prompt** (pestaña Prompt): opcionalmente cargar la plantilla recomendada y ajustar. Variables disponibles: `{nombre_bot}`, `{personalidad}`, `{tono}`, `{estilo_escritura}`, `{context}`, `{contexto_extra}`, `{question}`, `{nombre_empresa}`, `{productos}`, `{servicios}`.
7. **Probar el chat:** el agente detecta automáticamente su tenant y responde con grounding (si no hay dato relevante, responde “No tengo esa información”).

Cada agente = su propio tenant Weaviate (`agente_<id>`). El conocimiento de un agente **no** se filtra a otro. Esta arquitectura es transversal: sirve para cualquier empresa/agente sin cambios de código.

D.1. Cotizador médico (opcional, caso Vida Buena)

Si el nodo debe correr el cotizador médico:

1. Cargar los datos del tarifario (planes/tarifas/coberturas) — en el servidor de referencia se hizo con el comando `import_excel_vidabuena` a partir del Excel oficial. Los modelos (`Plan`, `Tarifa`, `Cobertura`, etc.) son parametrizables por empresa.

2. El **cotizador web** queda disponible en `http://<IP_0_DOMINIO>/cotizador/`.
 - Nota: probar con el `Host` correcto (`<IP_0_DOMINIO>`); un `Host` que no matchee el `server_name` cae al `server default` de nginx (404).
3. La **herramienta cotizar_plan** se activa automáticamente en el chat si la empresa del agente tiene planes cargados.

E. Checklist de verificación post-instalación

Marcar cada punto tras instalar:

- ☐ `python -c "import numpy; print(numpy.__version__)"` → **1.26.x** (no 2.x).
- ☐ `pip show langchain-core` → **0.3.67**; `langchain-openai` → **0.2.14**; `openai` → **1.93.0**.
- ☐ `pip show weaviate-client` → **4.21.0**.
- ☐ PostgreSQL accesible y `python manage.py migrate` sin errores.
- ☐ `systemctl status fastchatdj.service` → **active (running)**; socket `/run/fastchatdj.sock` presente.
- ☐ `nginx -t` OK y sitio accesible por HTTP.
- ☐ `docker compose ps` (en `/home/weaviate`) → contenedor `weaviate` **Up**.
- ☐ `curl -s -H "Authorization: Bearer <API_KEY>" http://127.0.0.1:8080/v1/.well-known/ready -o /dev/null -w "%{http_code}"` → **200**.
- ☐ Weaviate **no** responde en la IP pública (solo `127.0.0.1`).
- ☐ `/etc/fastchatdj.env` con permisos `600` y `WEAVIATE_API_KEY` coincidente con el compose.
- ☐ `credenciales.json` con permisos `600`.
- ☐ En el panel: crear un agente → verificar que se creó su tenant (`weaviate_rag.contar(<agente_id>)` disponible internamente).
- ☐ Subir un documento de prueba en Conocimiento → mensaje “Documento subido e indexado” y el nº de fragmentos sube.
- ☐ En Configuración del agente: el dropdown de **Modelo** trae la **lista viva** del proveedor.
- ☐ Probar el chat: pregunta sobre el documento subido → responde con el dato; pregunta fuera del conocimiento → “No tengo esa información”.



(Cotizador) `http://<IP_0_DOMINIO>/cotizador/` responde y las primas coinciden con la fuente oficial.

Apéndice — Comandos útiles de operación

```
# Reiniciar la app tras un cambio de código
sudo systemctl restart fastchatdj.service

# Logs de la app
journalctl -u fastchatdj.service -f

# Weaviate: estado, logs, reinicio
cd /home/weaviate && docker compose ps
docker compose logs -f weaviate
docker compose restart weaviate

# Verificar la conexión Weaviate desde el venv de la app (shell Django)
source /home/venv/bin/activate && cd /home/fastchatdj
python manage.py shell -c "from agents_ai import weaviate_rag;
    print(weaviate_rag.contar(<agente_id>))"
```